

Exercise Sheet 8

Lab Sessions: June 25/26, 2026

1 Preparation

1.1 Repository Update

For this exercise, we provide files in the [RepEng FIMGit Repository](#) (directory `LabSession8/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

1.2 DuckDB Setup

In this exercise, we are using DuckDB. Similarly to SQLite, DuckDB is an embedded database. However, while SQLite is a row-oriented database optimized for transactional processing (OLTP), DuckDB is a column-oriented database optimized for analytical queries (OLAP).

Spin up a Docker container running DuckDB by following these steps, depending on whether you are running Docker on your own machine or on the server (narrow-sea):

Local Setup

Navigate to `LabSession8/duckdb-local` in the FIMGit Repository and spin up the container using `docker compose`:

Terminal Session

bash

```
user@host$ cd LabSession8/duckdb-local
user@host$ docker compose up -d
```

You should now be able to access the DuckDB GUI via this link: <http://localhost:4213>

Server Setup (narrow-sea)

On narrow-sea, you need to make sure that you choose a unique port number, so there are no conflicts with containers of other students. The command `get_my_port` assigns you a unique port number.

Navigate to `LabSession8/duckdb-narrow-sea` in the FIMGit Repository and spin up the container using `docker compose` as follows:.

Terminal Session

bash

```
user@host$ cd LabSession8/duckdb-narrow-sea
user@host$ MY_PORT=$(get_my_port) docker compose up -d
```

After running these commands, your DuckDB container is running on the server and available on your assigned port. However, DuckDB expects you to connect to the GUI on localhost. Hence, we will use an SSH tunnel, to allow connecting via localhost. Execute the following command in terminal session on your local machine (not on the server!):

Terminal Session

bash

```
user@host$ ssh -L <PORT>:localhost:<PORT> \
    <FIM-ACCOUNT>@narrow-sea.sdb.s.fim.uni-passau.de
```

Note: In Windows Powershell, replace the trailing \ with a `.

Replace <PORT> with your unique assigned port (obtained by running `get_my_port` on the server) and <FIM-ACCOUNT> with your FIM account name.

As long as the SSH connection established with the command above remains open, you should be able to reach the DuckDB GUI at <http://localhost:<PORT>>.

1.3 Using the DuckDB GUI

In the DuckDB GUI, you can create databases on the left-hand sidebar by clicking “+” next to “Attached databases”.

In the same sidebar, you can create Notebooks in which you can write your SQL queries, by clicking the “+” next to “Notebooks”.

Hint: The DuckDB UI does not persist its configuration across container restarts. While your database files remain safely stored in the Docker volume, any attached databases will disappear from the sidebar after a restart. To re-acquire access, click the “+” icon and re-attach them using the exact same file path you specified initially.

Furthermore, accessing saved notebooks after a restart is currently quite inconvenient. We therefore recommend saving your SQL queries outside the DuckDB UI for later reference.

2 Data Restructuring: Tidy Data

The following exercise is adapted from the O’Reilly book “*R for Data Science*” (1st Edition) (Ch. 12) by Hadley Wickham and Garrett Grolemund¹.

In tidy data, each variable is stored in its own column, each observation in its own row, and each value in its own cell. This task will cover tidying your data through restructuring.

¹ Available online at <https://r4ds.had.co.nz/tidy-data.html>.

2.1 Creating the countries Database

Create a new database `countries`, to be used in this task.

2.2 Pivoting

Pivoting is one of the most common steps in tidying data allowing better data organization and analysis. DuckDB supports the `PIVOT` function to pivot rows to columns, and the `UNPIVOT` function to pivot columns to rows. Details on the [PIVOT](https://duckdb.org/docs/current/sql/statements/pivot)² and [UNPIVOT](https://duckdb.org/docs/current/sql/statements/unpivot)³ functions are available in the DuckDB documentation.

2.2.1 Pivoting Rows to Columns

Import the table `countries_long` using the file `countries_long.sql` available in the FIMGit repository, into the database `countries`. The table `countries_long`, an excerpt of which is shown below, manages the population and tuberculosis cases in a country in certain years.

countries_long			
country	year	type	count
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
...	...	...	...

Explain why this table is not tidy. Then, write SQL queries that pivot the data so that it conforms to the rules of tidy data,

- (a) using `PIVOT`,
- (b) without using `PIVOT`.

2.2.2 Pivoting Columns to Rows

Import the table `countries_wide` using the file `countries_wide.sql`, available in the FIMGit repository, into the database `countries`. The table `countries_wide`, shown below, describes the number of tuberculosis cases in a country in certain years.

²<https://duckdb.org/docs/current/sql/statements/pivot>

³<https://duckdb.org/docs/current/sql/statements/unpivot>

countries_wide		
country	year_1999	year_2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

Explain why this table does not conform to the principles of tidy data. Then, write SQL queries that pivot the data so that it conforms to the rules of tidy data,

- (a) using `UNPIVOT`,
- (b) without using `UNPIVOT`.

2.3 Splitting

Import the table `countries_rate` using the file `countries_rate.sql`, available in the FIMGit repository, into the database `countries`. Table `countries_rate`, below shows the rate of tuberculosis cases (given as the number of cases and the population) in a country, by year.

countries_rate		
country	year	rate
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583

Explain why this table does not conform to the principles of tidy data. Then create a new table with properly separated columns that follows the principles of tidy data. Further, separate the column `year` into two columns, `century` and `year_within_century`.

Hint: You may use some of the following functions to split String values:

- `LEFT`
- `STRPOS`
- `SUBSTRING`

2.4 Concatenation

Consider the table obtained by separating columns in the previous task. Unite the columns `century` and `year_within_century` back into a single column, called `year`.

2.5 Case Study

Import table `countries_who` using the file `countries_who.sql`, available in the FIMGit repository, into the database `countries`. The table represents tuberculosis cases described in the *2014 World Health Organization Global Tuberculosis Report* and is a shortened version of the table accompanying the book that this exercise is built on⁴. An excerpt of the table is shown below. It describes cases in countries by year for different scenarios (e.g., `new_sp_m014` – see below for details on these column names).

countries_who						
country	iso2	iso3	year	new_sp_m014	...	new_sp_f65
Azerbaijan	AZ	AZE	1994	NULL	...	NULL
Azerbaijan	AZ	AZE	1995	0	...	0
Azerbaijan	AZ	AZE	1996	0	...	10
...	...	...	...	...	...	...

Transform the table into a tidy state.

Hint: Columns such as `new_sp_m014` describe cases. Their names have the following pattern:

`<new/old>_<type>_<sex><age group>`, where `<new/old>` describes whether column contains new or old cases of tuberculosis, `<type>` describes the type of tuberculosis, `<sex>` describes the biological sex (f/m), and `<age group>` describes the age group, e.g., “014” stands for ages 0 – 14.

You do not need to change any of the values to be more consumable, e.g., when transforming `<age group>` to a distinct column, you may keep the value “014”.

⁴<https://github.com/tidyverse/tidyr/blob/main/data-raw/who.csv>

3 Discretization, Binarization, and Dummy Variables

In this task, we apply common transformations to the table `countries_who_tidy`, which results from the “Case Study” in Task 2.5. You may either use the result you obtained in Task 2.5 or import the table `countries_who_tidy` using file `countries_who_tidy.sql`, available in the FIMGit repository.

3.1 Discretization

Classify the number of tuberculosis cases to get a simplified overview of severity.

Write a SQL query that selects all columns from `countries_who_tidy` and adds a new column `severity_level`. Map the numeric *value* (number of cases) to the following buckets:

- “Low” if the value is less than 50.
- “Medium” if the value is between 50 (inclusive) and 500 (exclusive).
- “High” if the value is 500 or more.

3.2 Binarization and Dummy Variables

Certain algorithms require numerical input and cannot handle categorical strings directly. To prepare data accordingly, transform the demographic columns into numeric representations using the following techniques:

- Binarization:** Create a binary column `is_adult`. The value should be 1 if the `age_group` is not “014” (indicating the person is older than 14), and 0 otherwise.
- Dummy Variables (One-Hot Encoding):** Convert the categorical column `sex` into numerical columns. Create two new columns, `sex_m` and `sex_f`. `sex_m` should contain 1 if the sex is “m” and 0 otherwise (vice versa for `sex_f`).

Write a SQL query that returns the original columns of `countries_who_tidy` as well as the derived columns described above.

4 Tidy Data (Multiple-Choice, Antwort-Wahl-Verfahren)

In this task, each question has exactly one correct answer. Mark the correct answer.

1. The following table captures food preferences of cats.

cat_food_preference		
cat	food_brand	rating
Luna	Whiskas	4.3
Felix	Sheba	4.8
Momo	Felix	3.5
Nala	Whiskas	5.0
Simba	Purina	2.3
Tiger	Sheba	1.3

We perform the following changes:

```
ALTER TABLE CatFoodPreference ADD COLUMN PreferenceCategory VARCHAR(8);
```

```
UPDATE CatFoodPreference
SET PreferenceCategory = CASE WHEN Rating > 4.5 THEN 'loves'
                              WHEN Rating > 2.5 THEN 'likes'
                              WHEN Rating > 0.5 THEN 'dislikes'
                              ELSE 'hates' END;
```

What is the term that best describes this transformation?

- ☐ Discretization ☐ Creating dummy values ☐ Normalization
☐ Pivoting ☐ Binarization ☐ None of these options

2. Now, consider the following table that differentiates between dry and wet cat food:

cat_food_preference_dry_wet			
cat	food_brand	rating_dry	rating_wet
Luna	Whiskas	4.3	3.8
Felix	Sheba	4.8	4.2
Momo	Felix	3.5	4.0
Nala	Whiskas	5.0	4.7
Simba	Purina	2.3	3.1
Tiger	Sheba	1.3	2.5

Is the resulting table tidy?

- ☐ Yes ☐ No